

## PART 3 REACT NATIVE COURSE



# Part 3: Advanced Features (Advanced Level)

You're absolutely right! Let me provide you with comprehensive textual content along with the code examples for Part 3.

## Chapter 9: Native Device Features and Permissions

### Understanding Native Device Integration

React Native's power lies in its ability to access native device features while maintaining a single codebase. This chapter covers how to integrate with device hardware like camera, location services, storage, and sensors. Understanding these features is crucial for building modern mobile applications that feel truly native.

#### Why Device Features Matter:

- **Enhanced User Experience:** Access to camera, location, and sensors creates immersive experiences
- **Offline Capabilities:** Local storage enables apps to work without internet connectivity
- **Hardware Optimization:** Leverage device-specific capabilities for better performance
- **Native Feel:** Users expect mobile apps to integrate seamlessly with their device

### Camera Integration

The camera is one of the most commonly used device features in mobile apps. React Native Expo provides excellent camera integration through the `expo-camera` library.

#### Key Camera Concepts:

- **Permissions:** Always request camera permissions before accessing
- **Camera Types:** Front-facing vs back-facing cameras
- **Media Capture:** Photos vs videos with different quality settings
- **Real-time Preview:** Live camera feed display

```
import React, { useState, useEffect, useRef } from 'react';
import {
  View,
  Text,
  TouchableOpacity,
  Alert,
  Image,
  StyleSheet,
} from 'react-native';
import { Camera } from 'expo-camera';
import * as MediaLibrary from 'expo-media-library';

export default function CameraExample() {
```

 Copy

```

// State management for camera functionality
const [hasPermission, setHasPermission] = useState(null);
const [cameraType, setCameraType] = useState(Camera.Constants.Type.back);
const [capturedImage, setCapturedImage] = useState(null);
const [isRecording, setIsRecording] = useState(false);
const cameraRef = useRef(null);

// Request camera permissions when component mounts
useEffect(() => {
  requestPermissions();
}, []);

/**
 * Request necessary permissions for camera and media library access
 * This is a critical step - without permissions, camera features won't work
 */
const requestPermissions = async () => {
  try {
    // Request camera permission
    const cameraResult = await Camera.requestCameraPermissionsAsync();

    // Request media library permission (for saving photos)
    const mediaResult = await MediaLibrary.requestPermissionsAsync();

    // Both permissions must be granted for full functionality
    if (cameraResult.status === 'granted' && mediaResult.status === 'granted') {
      setHasPermission(true);
    } else {
      setHasPermission(false);
      Alert.alert(
        'Permissions Required',
        'Camera and media library access are required for this feature to work properly.'
      );
    }
  } catch (error) {
    console.error('Permission request failed:', error);
    setHasPermission(false);
  }
};

/**
 * Capture a photo using the camera
 * This function handles the entire photo capture workflow
 */
const takePicture = async () => {
  if (cameraRef.current) {
    try {
      // Configure photo options

```

```
const options = {
  quality: 0.8, // Image quality (0-1, where 1 is highest)
  base64: false, // Don't return base64 data (saves memory)
  exif: false, // Don't include EXIF data
};

// Capture the photo
const photo = await cameraRef.current.takePictureAsync(options);

// Update state with captured image
setCapturedImage(photo.uri);

// Optionally save to device gallery
await MediaLibrary.saveToLibraryAsync(photo.uri);

Alert.alert('Success', 'Photo captured and saved to gallery!');
} catch (error) {
  console.error('Failed to take picture:', error);
  Alert.alert('Error', 'Failed to capture photo. Please try again.');
```

```
};
```

```
/**
 * Toggle between front and back cameras
 * Most devices have multiple cameras, this allows switching
 */
```

```
const toggleCameraType = () => {
  setCameraType(
    cameraType === Camera.Constants.Type.back
      ? Camera.Constants.Type.front
      : Camera.Constants.Type.back
  );
};
```

```
/**
 * Start video recording
 * Video recording requires different handling than photo capture
 */
```

```
const startRecording = async () => {
  if (cameraRef.current && !isRecording) {
    try {
      setIsRecording(true);

      const videoOptions = {
        quality: Camera.Constants.VideoQuality['720p'],
        maxDuration: 60, // Maximum 60 seconds
      };
    }
  }
};
```

```

const video = await cameraRef.current.recordAsync(videoOptions);

// Save video to gallery
await MediaLibrary.saveToLibraryAsync(video.uri);

Alert.alert('Success', 'Video recorded and saved!');
} catch (error) {
  console.error('Recording failed:', error);
  Alert.alert('Error', 'Failed to record video');
} finally {
  setIsRecording(false);
}
};

/**
 * Stop video recording
 */
const stopRecording = () => {
  if (cameraRef.current && isRecording) {
    cameraRef.current.stopRecording();
  }
};

// Handle permission states
if (hasPermission === null) {
  return (
    <View style={styles.centerContainer}>
      <Text>Requesting camera permissions...</Text>
    </View>
  );
}

if (hasPermission === false) {
  return (
    <View style={styles.centerContainer}>
      <Text style={styles.noAccessText}>
        Camera access denied. Please enable camera permissions in your device
settings.
      </Text>
      <TouchableOpacity style={styles.button} onPress={requestPermissions}>
        <Text style={styles.buttonText}>Request Permissions Again</Text>
      </TouchableOpacity>
    </View>
  );
}

return (
  <View style={styles.container}>

```

```

    {/* Camera Preview */}
    <Camera
      style={styles.camera}
      type={cameraType}
      ref={cameraRef}
    >
      <View style={styles.cameraOverlay}>
        {/* Camera Controls */}
        <View style={styles.topControls}>
          <TouchableOpacity style={styles.controlButton} onPress=
{toggleCameraType}>
            <Text style={styles.controlButtonText}>
              {cameraType === Camera.Constants.Type.back ? '📷' : '📹'}
            </Text>
          </TouchableOpacity>
        </View>

        <View style={styles.bottomControls}>
          {/* Photo Capture Button */}
          <TouchableOpacity style={styles.captureButton} onPress={takePicture}>
            <View style={styles.captureButtonInner} />
          </TouchableOpacity>

          {/* Video Recording Button */}
          <TouchableOpacity
            style={[styles.recordButton, isRecording && styles.recordingButton]}
            onPress={isRecording ? stopRecording : startRecording}
          >
            <Text style={styles.controlButtonText}>
              {isRecording ? '🛑' : '📹'}
            </Text>
          </TouchableOpacity>
        </View>
      </View>
    </Camera>

    {/* Captured Image Preview */}
    {capturedImage && (
      <View style={styles.imagePreview}>
        <Text style={styles.previewTitle}>Last Captured Photo:</Text>
        <Image source={{ uri: capturedImage }} style={styles.previewImage} />
      </View>
    )}
  </View>
);
}

const styles = StyleSheet.create({
  container: {

```

```
    flex: 1,
    backgroundColor: '#000',
  },
  centerContainer: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 20,
  },
  noAccessText: {
    fontSize: 16,
    textAlign: 'center',
    marginBottom: 20,
    color: '#666',
  },
  camera: {
    flex: 1,
  },
  cameraOverlay: {
    flex: 1,
    backgroundColor: 'transparent',
    justifyContent: 'space-between',
  },
  topControls: {
    flexDirection: 'row',
    justifyContent: 'flex-end',
    padding: 20,
    paddingTop: 50,
  },
  bottomControls: {
    flexDirection: 'row',
    justifyContent: 'space-around',
    alignItems: 'center',
    padding: 30,
    paddingBottom: 50,
  },
  controlButton: {
    width: 50,
    height: 50,
    borderRadius: 25,
    backgroundColor: 'rgba(255,255,255,0.3)',
    justifyContent: 'center',
    alignItems: 'center',
  },
  controlButtonText: {
    fontSize: 24,
  },
  captureButton: {
    width: 80,
```

```
    height: 80,
    borderRadius: 40,
    backgroundColor: 'rgba(255,255,255,0.3)',
    justifyContent: 'center',
    alignItems: 'center',
    borderWidth: 4,
    borderColor: 'white',
  },
  captureButtonInner: {
    width: 60,
    height: 60,
    borderRadius: 30,
    backgroundColor: 'white',
  },
  recordButton: {
    width: 60,
    height: 60,
    borderRadius: 30,
    backgroundColor: 'rgba(255,0,0,0.8)',
    justifyContent: 'center',
    alignItems: 'center',
  },
  recordingButton: {
    backgroundColor: 'rgba(255,0,0,1)',
  },
  button: {
    backgroundColor: '#3498db',
    paddingHorizontal: 20,
    paddingVertical: 10,
    borderRadius: 8,
  },
  buttonText: {
    color: 'white',
    fontSize: 16,
    fontWeight: '600',
  },
  imagePreview: {
    position: 'absolute',
    bottom: 20,
    left: 20,
    backgroundColor: 'rgba(0,0,0,0.8)',
    padding: 10,
    borderRadius: 8,
  },
  previewTitle: {
    color: 'white',
    fontSize: 12,
    marginBottom: 5,
  },
},
```

```
    previewImage: {
      width: 100,
      height: 100,
      borderRadius: 8,
    },
  });
```

## Location Services

Location services enable apps to provide location-aware features like maps, weather updates, and location-based recommendations. Understanding how to work with GPS coordinates, geocoding, and location permissions is essential for many modern apps.

### Location Service Concepts:

- **GPS Accuracy:** Different accuracy levels affect battery usage
- **Foreground vs Background:** Location tracking permissions differ
- **Geocoding:** Converting coordinates to addresses and vice versa
- **Location Updates:** Real-time vs one-time location requests

```
import React, { useState, useEffect } from 'react';
import {
  View,
  Text,
  TouchableOpacity,
  Alert,
  ActivityIndicator,
  StyleSheet,
} from 'react-native';
import * as Location from 'expo-location';

export default function LocationExample() {
  // State management for location functionality
  const [location, setLocation] = useState(null);
  const [address, setAddress] = useState(null);
  const [loading, setLoading] = useState(false);
  const [watchingLocation, setWatchingLocation] = useState(false);
  const [locationSubscription, setLocationSubscription] = useState(null);

  /**
   * Request location permissions and get initial location
   * Location permissions are sensitive and require careful handling
   */
  useEffect(() => {
    requestLocationPermission();
    return () => {
      // Cleanup location watching when component unmounts
    };
  }, []);
```

 Copy

```
    if (locationSubscription) {
      locationSubscription.remove();
    }
  };
}, []);

/**
 * Request location permissions
 * Different permission levels provide different capabilities
 */
const requestLocationPermission = async () => {
  try {
    // Request foreground location permission
    const { status } = await Location.requestForegroundPermissionsAsync();

    if (status !== 'granted') {
      Alert.alert(
        'Permission Denied',
        'Location permission is required to use this feature. Please enable it in your device settings.'
      );
      return;
    }

    // Optionally request background location permission
    // Note: This requires additional configuration and App Store approval
    const backgroundStatus = await Location.requestBackgroundPermissionsAsync();
    if (backgroundStatus.status === 'granted') {
      console.log('Background location permission granted');
    }

  } catch (error) {
    console.error('Location permission error:', error);
    Alert.alert('Error', 'Failed to request location permissions');
  }
};

/**
 * Get current location (one-time request)
 * This provides a snapshot of the user's current position
 */
const getCurrentLocation = async () => {
  try {
    setLoading(true);

    // Configure location options
    const locationOptions = {
      accuracy: Location.Accuracy.High, // High accuracy uses GPS
      timeout: 15000, // 15 second timeout
    };
  }
};
```

```

    maximumAge: 10000, // Use cached location if less than 10 seconds old
  };

  // Get current position
  const currentLocation = await
Location.getCurrentPositionAsync(locationOptions);

  setLocation(currentLocation);

  // Get address from coordinates (reverse geocoding)
  await getAddressFromCoordinates(
    currentLocation.coords.latitude,
    currentLocation.coords.longitude
  );

} catch (error) {
  console.error('Location error:', error);

  if (error.code === 'LOCATION_UNAVAILABLE') {
    Alert.alert('Error', 'Location services are disabled. Please enable them
and try again.');
```

```

  } else if (error.code === 'PERMISSION_DENIED') {
    Alert.alert('Error', 'Location permission denied');
```

```

  } else if (error.code === 'TIMEOUT') {
    Alert.alert('Error', 'Location request timed out. Please try again.');
```

```

  } else {
    Alert.alert('Error', 'Failed to get location. Please try again.');
```

```

  }
} finally {
  setLoading(false);
}
};

/**
 * Start watching location changes
 * This provides continuous location updates as the user moves
 */
const startWatchingLocation = async () => {
  try {
    setWatchingLocation(true);

    const watchOptions = {
      accuracy: Location.Accuracy.High,
      timeInterval: 5000, // Update every 5 seconds
      distanceInterval: 10, // Update when moved 10 meters
    };

    // Start location watching
    const subscription = await Location.watchPositionAsync(
```

```

        watchOptions,
        (newLocation) => {
            setLocation(newLocation);
            console.log('Location updated:', newLocation.coords);
        }
    );

    setLocationSubscription(subscription);

} catch (error) {
    console.error('Watch location error:', error);
    Alert.alert('Error', 'Failed to start location tracking');
    setWatchingLocation(false);
}
};

/**
 * Stop watching location changes
 */
const stopWatchingLocation = () => {
    if (locationSubscription) {
        locationSubscription.remove();
        setLocationSubscription(null);
    }
    setWatchingLocation(false);
};

/**
 * Convert coordinates to human-readable address
 * This is called reverse geocoding
 */
const getAddressFromCoordinates = async (latitude, longitude) => {
    try {
        const reverseGeocode = await Location.reverseGeocodeAsync({
            latitude,
            longitude,
        });

        if (reverseGeocode.length > 0) {
            const addr = reverseGeocode[0];
            const fullAddress = `${addr.streetNumber || ''} ${addr.street || ''},
${addr.city || ''}, ${addr.region || ''} ${addr.postalCode || ''}, ${addr.country
|| ''}`.trim();
            setAddress(fullAddress);
        }
    } catch (error) {
        console.error('Reverse geocoding error:', error);
        setAddress('Address not available');
    }
}

```

```

};

/**
 * Calculate distance between two points
 * Useful for distance-based features
 */
const calculateDistance = (lat1, lon1, lat2, lon2) => {
  const R = 6371; // Earth's radius in kilometers
  const dLat = (lat2 - lat1) * Math.PI / 180;
  const dLon = (lon2 - lon1) * Math.PI / 180;
  const a =
    Math.sin(dLat/2) * Math.sin(dLat/2) +
    Math.cos(lat1 * Math.PI / 180) * Math.cos(lat2 * Math.PI / 180) *
    Math.sin(dLon/2) * Math.sin(dLon/2);
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));
  return R * c; // Distance in kilometers
};

return (
  <View style={styles.container}>
    <Text style={styles.title}>Location Services Demo</Text>

    {/* Current Location Display */}
    <View style={styles.infoContainer}>
      <Text style={styles.infoTitle}>Current Location:</Text>
      {location ? (
        <View>
          <Text style={styles.infoText}>
            Latitude: {location.coords.latitude.toFixed(6)}
          </Text>
          <Text style={styles.infoText}>
            Longitude: {location.coords.longitude.toFixed(6)}
          </Text>
          <Text style={styles.infoText}>
            Accuracy: ±{location.coords.accuracy.toFixed(0)} meters
          </Text>
          <Text style={styles.infoText}>
            Altitude: {location.coords.altitude?.toFixed(0) || 'N/A'} meters
          </Text>
          <Text style={styles.infoText}>
            Speed: {location.coords.speed ? `${(location.coords.speed *
3.6).toFixed(1)} km/h` : 'N/A'}
          </Text>
          <Text style={styles.infoText}>
            Timestamp: {new Date(location.timestamp).toLocaleString()}
          </Text>
        </View>
      ) : (
        <Text style={styles.infoText}>Location not available</Text>

```

```

    })
</View>

{/* Address Display */}
{address && (
  <View style={styles.infoContainer}>
    <Text style={styles.infoTitle}>Address:</Text>
    <Text style={styles.addressText}>{address}</Text>
  </View>
)}

{/* Control Buttons */}
<View style={styles.buttonContainer}>
  <TouchableOpacity
    style={[styles.button, loading && styles.disabledButton]}
    onPress={getCurrentLocation}
    disabled={loading}
  >
    {loading ? (
      <ActivityIndicator color="white" size="small" />
    ) : (
      <Text style={styles.buttonText}>Get Current Location</Text>
    )}
  </TouchableOpacity>

  <TouchableOpacity
    style={[styles.button, watchingLocation ? styles.stopButton :
styles.startButton]}
    onPress={watchingLocation ? stopWatchingLocation :
startWatchingLocation}
  >
    <Text style={styles.buttonText}>
      {watchingLocation ? 'Stop Tracking' : 'Start Location Tracking'}
    </Text>
  </TouchableOpacity>
</View>

{/* Status Indicator */}
{watchingLocation && (
  <View style={styles.statusContainer}>
    <View style={styles.statusIndicator} />
    <Text style={styles.statusText}>Location tracking active</Text>
  </View>
)}
</View>
);
}

```

```
const styles = StyleSheet.create({
```

```
container: {
  flex: 1,
  padding: 20,
  backgroundColor: '#f8f9fa',
},
title: {
  fontSize: 24,
  fontWeight: 'bold',
  textAlign: 'center',
  marginBottom: 30,
  color: '#2c3e50',
},
infoContainer: {
  backgroundColor: 'white',
  padding: 15,
  borderRadius: 8,
  marginBottom: 15,
  elevation: 2,
  shadowColor: '#000',
  shadowOffset: { width: 0, height: 2 },
  shadowOpacity: 0.1,
  shadowRadius: 4,
},
infoTitle: {
  fontSize: 18,
  fontWeight: 'bold',
  marginBottom: 10,
  color: '#2c3e50',
},
infoText: {
  fontSize: 14,
  color: '#34495e',
  marginBottom: 5,
},
addressText: {
  fontSize: 14,
  color: '#34495e',
  fontStyle: 'italic',
},
buttonContainer: {
  marginTop: 20,
},
button: {
  backgroundColor: '#3498db',
  padding: 15,
  borderRadius: 8,
  alignItems: 'center',
  marginBottom: 10,
},
```

```
disabledButton: {
  backgroundColor: '#bdc3c7',
},
startButton: {
  backgroundColor: '#2ecc71',
},
stopButton: {
  backgroundColor: '#e74c3c',
},
buttonText: {
  color: 'white',
  fontSize: 16,
  fontWeight: '600',
},
statusContainer: {
  flexDirection: 'row',
  alignItems: 'center',
  justifyContent: 'center',
  marginTop: 20,
},
statusIndicator: {
  width: 10,
  height: 10,
  borderRadius: 5,
  backgroundColor: '#2ecc71',
  marginRight: 8,
},
statusText: {
  fontSize: 14,
  color: '#2ecc71',
  fontWeight: '600',
},
});
```

## Local Storage with AsyncStorage

AsyncStorage is React Native's key-value storage system that persists data between app sessions. Understanding how to properly store and retrieve data locally is crucial for creating apps that work offline and provide a smooth user experience.

### AsyncStorage Key Concepts:

- **Asynchronous Operations:** All storage operations return promises
- **String Storage:** Only strings can be stored (objects must be serialized)
- **Persistence:** Data survives app restarts and device reboots
- **Platform Differences:** Implementation varies between iOS and Android



```
import React, { useState, useEffect } from 'react';
import {
  View,
  Text,
  TextInput,
  TouchableOpacity,
  FlatList,
  Alert,
  StyleSheet,
} from 'react-native';
import AsyncStorage from '@react-native-async-storage/async-storage';

export default function StorageExample() {
  // State for managing notes and UI
  const [notes, setNotes] = useState([]);
  const [newNote, setNewNote] = useState('');
  const [editingId, setEditingId] = useState(null);
  const [editText, setEditText] = useState('');
  const [storageInfo, setStorageInfo] = useState({});

  // Storage keys - using consistent naming convention
  const STORAGE_KEYS = {
    NOTES: '@notes_storage',
    USER_PREFERENCES: '@user_preferences',
    APP_DATA: '@app_data',
  };

  /**
   * Load data when component mounts
   * This ensures data persistence across app sessions
   */
  useEffect(() => {
    loadNotesFromStorage();
    getStorageInfo();
  }, []);

  /**
   * Load notes from AsyncStorage
   * Handle parsing errors gracefully to prevent app crashes
   */
  const loadNotesFromStorage = async () => {
    try {
      const storedNotes = await AsyncStorage.getItem(STORAGE_KEYS.NOTES);

      if (storedNotes !== null) {
        // Parse JSON data and update state
        const parsedNotes = JSON.parse(storedNotes);
        setNotes(parsedNotes);
      }
    } catch (error) {
      console.log('Error loading notes from storage:', error);
    }
  };
}
```

```
    } else {
      // No data found - this is normal for first app launch
      console.log('No notes found in storage');
    }
  } catch (error) {
    console.error('Error loading notes:', error);
    Alert.alert(
      'Storage Error',
      'Failed to load notes from storage. Starting with empty list.'
    );
  }
};

/**
 * Save notes to AsyncStorage
 * Always stringify objects before storing
 */
const saveNotesToStorage = async (notesToSave) => {
  try {
    const jsonNotes = JSON.stringify(notesToSave);
    await AsyncStorage.setItem(STORAGE_KEYS.NOTES, jsonNotes);
  } catch (error) {
    console.error('Error saving notes:', error);
    Alert.alert('Storage Error', 'Failed to save notes to storage');
  }
};

/**
 * Add a new note
 * Demonstrates creating and persisting new data
 */
const addNote = async () => {
  if (newNote.trim().length === 0) {
    Alert.alert('Validation Error', 'Please enter a note');
    return;
  }

  const note = {
    id: Date.now().toString(), // Simple ID generation
    text: newNote.trim(),
    createdAt: new Date().toISOString(),
    updatedAt: new Date().toISOString(),
  };

  const updatedNotes = [note, ...notes];
  setNotes(updatedNotes);
  setNewNote('');

  // Persist to storage

```

```
    await saveNotesToStorage(updatedNotes);
  };

  /**
   * Update an existing note
   * Demonstrates data modification and persistence
   */
  const updateNote = async (id, newText) => {
    if (newText.trim().length === 0) {
      Alert.alert('Validation Error', 'Note cannot be empty');
      return;
    }

    const updatedNotes = notes.map(note =>
      note.id === id
        ? {
            ...note,
            text: newText.trim(),
            updatedAt: new Date().toISOString(),
          }
        : note
    );

    setNotes(updatedNotes);
    setEditingId(null);
    setEditText('');

    // Persist changes
    await saveNotesToStorage(updatedNotes);
  };

  /**
   * Delete a note
   * Demonstrates data removal and cleanup
   */
  const deleteNote = async (id) => {
    Alert.alert(
      'Confirm Delete',
      'Are you sure you want to delete this note?',
      [
        { text: 'Cancel', style: 'cancel' },
        {
          text: 'Delete',
          style: 'destructive',
          onPress: async () => {
            const updatedNotes = notes.filter(note => note.id !== id);
            setNotes(updatedNotes);

            // Update storage

```

```
        await saveNotesToStorage(updatedNotes);
    }
}
]
);
};

/**
 * Clear all notes
 * Demonstrates bulk data removal
 */
const clearAllNotes = async () => {
    Alert.alert(
        'Clear All Notes',
        'This will delete all your notes. This action cannot be undone.',
        [
            { text: 'Cancel', style: 'cancel' },
            {
                text: 'Clear All',
                style: 'destructive',
                onPress: async () => {
                    setNotes([]);
                    await AsyncStorage.removeItem(STORAGE_KEYS.NOTES);
                }
            }
        ]
    );
};

/**
 * Get storage usage information
 * Helpful for debugging and storage management
 */
const getStorageInfo = async () => {
    try {
        // Get all keys in storage
        const keys = await AsyncStorage.getAllKeys();

        // Calculate approximate storage usage
        let totalSize = 0;
        const keyInfo = {};

        for (const key of keys) {
            const value = await AsyncStorage.getItem(key);
            const size = value ? value.length : 0;
            totalSize += size;
            keyInfo[key] = size;
        }
    }
};
```

```

    setStorageInfo({
      totalKeys: keys.length,
      totalSize: totalSize,
      keys: keyInfo,
    });
  } catch (error) {
    console.error('Error getting storage info:', error);
  }
};

/**
 * Export/backup notes
 * Demonstrates data export functionality
 */
const exportNotes = async () => {
  if (notes.length === 0) {
    Alert.alert('No Data', 'No notes to export');
    return;
  }

  try {
    const exportData = {
      notes: notes,
      exportDate: new Date().toISOString(),
      version: '1.0',
    };

    const jsonString = JSON.stringify(exportData, null, 2);

    // In a real app, you might save to device storage or share via email
    Alert.alert(
      'Export Data',
      `Notes exported successfully!\n\nData size: ${jsonString.length} characters\nNotes count: ${notes.length}`,
      [{ text: 'OK' }]
    );

    console.log('Exported data:', jsonString);
  } catch (error) {
    console.error('Export error:', error);
    Alert.alert('Export Error', 'Failed to export notes');
  }
};

/**
 * Start editing a note
 */
const startEditing = (note) => {
  setEditingId(note.id);

```

```

    setEditText(note.text);
  };

  /**
   * Cancel editing
   */
  const cancelEditing = () => {
    setEditingId(null);
    setEditText('');
  };

  /**
   * Render individual note item
   */
  const renderNote = ({ item }) => (
    <View style={styles.noteItem}>
      {editingId === item.id ? (
        // Edit mode
        <View style={styles.editContainer}>
          <TextInput
            style={styles.editInput}
            value={editText}
            onChangeText={setEditText}
            multiline
            autoFocus
          />
          <View style={styles.editActions}>
            <TouchableOpacity
              style={[styles.actionButton, styles.saveButton]}
              onPress={() => updateNote(item.id, editText)}
            >
              <Text style={styles.actionButtonText}>Save</Text>
            </TouchableOpacity>
            <TouchableOpacity
              style={[styles.actionButton, styles.cancelButton]}
              onPress={cancelEditing}
            >
              <Text style={styles.actionButtonText}>Cancel</Text>
            </TouchableOpacity>
          </View>
        </View>
      ) : (
        // Display mode
        <View>
          <Text style={styles.noteText}>{item.text}</Text>
          <View style={styles.noteMetadata}>
            <Text style={styles.noteDate}>
              Created: {new Date(item.createdAt).toLocaleDateString()}
            </Text>
          </View>
        </View>
      )}
    </View>
  );

```

```

        {item.updatedAt !== item.createdAt && (
          <Text style={styles.noteDate}>
            Updated: {new Date(item.updatedAt).toLocaleDateString()}
          </Text>
        )}
      </View>
      <View style={styles.noteActions}>
        <TouchableOpacity
          style={[styles.actionButton, styles.editButton]}
          onPress={() => startEditing(item)}
        >
          <Text style={styles.actionButtonText}>Edit</Text>
        </TouchableOpacity>
        <TouchableOpacity
          style={[styles.actionButton, styles.deleteButton]}
          onPress={() => deleteNote(item.id)}
        >
          <Text style={styles.actionButtonText}>Delete</Text>
        </TouchableOpacity>
      </View>
    </View>
  )}
</View>
);

return (
  <View style={styles.container}>
    <Text style={styles.title}>AsyncStorage Demo - Notes App</Text>

    {/* Add Note Section */}
    <View style={styles.addNoteContainer}>
      <TextInput
        style={styles.addNoteInput}
        placeholder="Enter a new note..."
        value={newNote}
        onChangeText={setNewNote}
        multiline
      />
      <TouchableOpacity style={styles.addButton} onPress={addNote}>
        <Text style={styles.addButtonText}>Add Note</Text>
      </TouchableOpacity>
    </View>

    {/* Storage Info */}
    <View style={styles.storageInfo}>
      <Text style={styles.storageInfoText}>
        Notes: {notes.length} | Storage Keys: {storageInfo.totalKeys || 0} |
        Size: {storageInfo.totalSize || 0} chars
      </Text>
    </View>
  </View>
);

```

```

</View>

{/* Notes List */}
<FlatList
  data={notes}
  renderItem={renderNote}
  keyExtractor={({item}) => item.id}
  style={styles.notesList}
  contentContainerStyle={notes.length === 0 ? styles.emptyList : null}
  ListEmptyComponent={() => (
    <View style={styles.emptyContainer}>
      <Text style={styles.emptyText}>No notes yet</Text>
      <Text style={styles.emptySubtext}>Add your first note above</Text>
    </View>
  )}
/>

{/* Action Buttons */}
{notes.length > 0 && (
  <View style={styles.actionContainer}>
    <TouchableOpacity style={styles.exportButton} onPress={exportNotes}>
      <Text style={styles.actionButtonText}>Export Notes</Text>
    </TouchableOpacity>
    <TouchableOpacity style={styles.clearButton} onPress={clearAllNotes}>
      <Text style={styles.actionButtonText}>Clear All</Text>
    </TouchableOpacity>
  </View>
)}
</View>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    backgroundColor: '#f8f9fa',
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    textAlign: 'center',
    marginBottom: 20,
    color: '#2c3e50',
  },
  addNoteContainer: {
    backgroundColor: 'white',
    padding: 15,
    borderRadius: 8,

```

```
marginBottom: 15,
elevation: 2,
shadowColor: '#000',
shadowOffset: { width: 0, height: 2 },
shadowOpacity: 0.1,
shadowRadius: 4,
},
addNoteInput: {
  borderWidth: 1,
  borderColor: '#ddd',
  borderRadius: 8,
  padding: 12,
  fontSize: 16,
  marginBottom: 10,
  maxHeight: 100,
},
addButton: {
  backgroundColor: '#3498db',
  padding: 12,
  borderRadius: 8,
  alignItems: 'center',
},
addButtonText: {
  color: 'white',
  fontSize: 16,
  fontWeight: '600',
},
storageInfo: {
  backgroundColor: '#ecf0f1',
  padding: 10,
  borderRadius: 8,
  marginBottom: 15,
},
storageInfoText: {
  fontSize: 12,
  color: '#7f8c8d',
  textAlign: 'center',
},
notesList: {
  flex: 1,
},
emptyList: {
  flex: 1,
},
emptyContainer: {
  flex: 1,
  justifyContent: 'center',
  alignItems: 'center',
},
```

```
emptyText: {
  fontSize: 18,
  color: '#7f8c8d',
  marginBottom: 5,
},
emptySubtext: {
  fontSize: 14,
  color: '#bdc3c7',
},
noteItem: {
  backgroundColor: 'white',
  padding: 15,
  borderRadius: 8,
  marginBottom: 10,
  elevation: 2,
  shadowColor: '#000',
  shadowOffset: { width: 0, height: 2 },
  shadowOpacity: 0.1,
  shadowRadius: 4,
},
noteText: {
  fontSize: 16,
  color: '#2c3e50',
  lineHeight: 22,
  marginBottom: 10,
},
noteMetadata: {
  marginBottom: 10,
},
noteDate: {
  fontSize: 12,
  color: '#7f8c8d',
  fontStyle: 'italic',
},
noteActions: {
  flexDirection: 'row',
  justifyContent: 'flex-end',
},
editContainer: {
  flex: 1,
},
editInput: {
  borderWidth: 1,
  borderColor: '#3498db',
  borderRadius: 8,
  padding: 12,
  fontSize: 16,
  marginBottom: 10,
  maxHeight: 150,
```

```
    },
    editActions: {
      flexDirection: 'row',
      justifyContent: 'flex-end',
    },
    actionButton: {
      paddingHorizontal: 12,
      paddingVertical: 6,
      borderRadius: 4,
      marginLeft: 8,
    },
    editButton: {
      backgroundColor: '#f39c12',
    },
    deleteButton: {
      backgroundColor: '#e74c3c',
    },
    saveButton: {
      backgroundColor: '#2ecc71',
    },
    cancelButton: {
      backgroundColor: '#95a5a6',
    },
    actionButtonText: {
      color: 'white',
      fontSize: 12,
      fontWeight: '600',
    },
    actionContainer: {
      flexDirection: 'row',
      justifyContent: 'space-between',
      marginTop: 15,
    },
    exportButton: {
      backgroundColor: '#3498db',
      paddingHorizontal: 20,
      paddingVertical: 10,
      borderRadius: 8,
      flex: 0.48,
      alignItems: 'center',
    },
    clearButton: {
      backgroundColor: '#e74c3c',
      paddingHorizontal: 20,
      paddingVertical: 10,
      borderRadius: 8,
      flex: 0.48,
      alignItems: 'center',
    },
  },
```

```
});
```

This completes a comprehensive look at native device features in React Native. The key points covered include:

**Camera Integration:** Understanding how to request permissions, capture photos/videos, and handle different camera states with proper error handling.

**Location Services:** Implementing GPS functionality with different accuracy levels, handling permissions, reverse geocoding for addresses, and real-time location tracking.

**Local Storage:** Using AsyncStorage for persistent data storage, proper JSON serialization, error handling, and data management patterns that work reliably across app sessions.

Each example demonstrates real-world usage patterns with comprehensive error handling, user feedback, and best practices for mobile development. The code is heavily commented to explain not just what is happening, but why certain approaches are taken.

Ready for **Chapter 10: Animations and Advanced UI** when you want to continue!

# Part 3: Advanced Features (Advanced Level) - Continued

## Chapter 10: Animations and Advanced UI

### Understanding React Native Animations

Animations are crucial for creating engaging mobile experiences. React Native provides several animation APIs, each suited for different use cases. Understanding when and how to use each type is essential for building smooth, performant apps.

#### Animation Types in React Native:

- **Animated API:** JavaScript-driven animations with interpolation
- **LayoutAnimation:** Automatic animations for layout changes
- **Reanimated:** High-performance animations that run on the UI thread
- **Lottie:** Complex vector animations from After Effects

#### Performance Considerations:

- **UI Thread vs JS Thread:** Native animations perform better
- **useNativeDriver:** Moves animations to native thread for better performance
- **Interpolation:** Smooth value transitions between ranges
- **Gesture Integration:** Combining touch gestures with animations

```
import React, { useState, useRef, useEffect } from 'react';
import {
  View,
  Text,
  TouchableOpacity,
  Animated,
  PanGestureHandler,
  Dimensions,
  Easing,
  StyleSheet,
} from 'react-native';

const { width: SCREEN_WIDTH, height: SCREEN_HEIGHT } = Dimensions.get('window');

export default function AnimationsExample() {
  // Animation value references - these persist across re-renders
  const fadeValue = useRef(new Animated.Value(0)).current;
  const slideValue = useRef(new Animated.Value(0)).current;
  const scaleValue = useRef(new Animated.Value(1)).current;
  const rotateValue = useRef(new Animated.Value(0)).current;
```

 Copy

```

const colorValue = useRef(new Animated.Value(0)).current;

// Pan gesture animation
const panValue = useRef(new Animated.ValueXY({ x: 0, y: 0 })).current;

// Complex animation states
const [isAnimating, setIsAnimating] = useState(false);
const [animationType, setAnimationType] = useState('fade');

/**
 * Fade In/Out Animation
 * Basic opacity animation demonstrating timing and easing curves
 */
const fadeInOut = () => {
  // Determine target value based on current state
  const toValue = fadeValue._value === 0 ? 1 : 0;

  Animated.timing(fadeValue, {
    toValue: toValue,
    duration: 800, // Animation duration in milliseconds
    easing: Easing.inOut(Easing.ease), // Smooth easing curve
    useNativeDriver: true, // Use native driver for better performance
  }).start((finished) => {
    // Callback called when animation completes
    if (finished) {
      console.log('Fade animation completed');
    }
  });
};

/**
 * Slide Animation
 * Demonstrates position-based animations with interpolation
 */
const slideAnimation = () => {
  const toValue = slideValue._value === 0 ? 1 : 0;

  Animated.timing(slideValue, {
    toValue: toValue,
    duration: 600,
    easing: Easing.out(Easing.cubic),
    useNativeDriver: true,
  }).start();
};

/**
 * Scale Animation with Spring Physics
 * Spring animations feel more natural and responsive
 */

```

```
const scaleAnimation = () => {
  // Reset to original scale first
  scaleValue.setValue(1);

  // Create a spring animation sequence
  Animated.sequence([
    Animated.spring(scaleValue, {
      toValue: 1.5, // Scale up to 150%
      friction: 3, // Controls bounciness (lower = more bounce)
      tension: 100, // Controls speed (higher = faster)
      useNativeDriver: true,
    }),
    Animated.spring(scaleValue, {
      toValue: 1, // Return to original size
      friction: 3,
      tension: 100,
      useNativeDriver: true,
    }),
  ]).start();
};

/**
 * Rotation Animation
 * Demonstrates continuous rotation with interpolation
 */
const rotateAnimation = () => {
  // Reset rotation value
  rotateValue.setValue(0);

  Animated.timing(rotateValue, {
    toValue: 1,
    duration: 2000,
    easing: Easing.linear,
    useNativeDriver: true,
  }).start();
};

/**
 * Color Animation
 * Animating colors requires interpolation since colors aren't numbers
 */
const colorAnimation = () => {
  const toValue = colorValue._value === 0 ? 1 : 0;

  Animated.timing(colorValue, {
    toValue: toValue,
    duration: 1000,
    easing: Easing.inOut(Easing.ease),
    useNativeDriver: false, // Color animations can't use native driver
  }).start();
};
```

```
    }).start();
};

/**
 * Complex Staggered Animation
 * Multiple animations running in sequence or parallel
 */
const complexAnimation = () => {
  setIsAnimating(true);

  // Reset all values
  fadeValue.setValue(0);
  slideValue.setValue(0);
  scaleValue.setValue(1);
  rotateValue.setValue(0);

  // Create parallel animations that start at different times
  Animated.stagger(200, [
    Animated.timing(fadeValue, {
      toValue: 1,
      duration: 800,
      useNativeDriver: true,
    }),
    Animated.timing(slideValue, {
      toValue: 1,
      duration: 800,
      useNativeDriver: true,
    }),
    Animated.spring(scaleValue, {
      toValue: 1.2,
      friction: 4,
      useNativeDriver: true,
    }),
    Animated.timing(rotateValue, {
      toValue: 1,
      duration: 1000,
      useNativeDriver: true,
    }),
  ]).start(() => {
    setIsAnimating(false);
  });
};

/**
 * Gesture-based Animation
 * Draggable element that snaps back to center
 */
const onPanGestureEvent = Animated.event(
  [{ nativeEvent: { translationX: panValue.x, translationY: panValue.y } }],
```

```

    { useNativeDriver: false }
  );

const onPanHandlerStateChange = (event) => {
  if (event.nativeEvent.oldState === State.ACTIVE) {
    // Snap back to center when gesture ends
    Animated.spring(panValue, {
      toValue: { x: 0, y: 0 },
      friction: 5,
      useNativeDriver: false,
    }).start();
  }
};

/**
 * Loop Animation
 * Continuous animation that repeats indefinitely
 */
const startLoopAnimation = () => {
  const createLoopAnimation = () => {
    return Animated.loop(
      Animated.sequence([
        Animated.timing(scaleValue, {
          toValue: 1.3,
          duration: 1000,
          easing: Easing.inOut(Easing.ease),
          useNativeDriver: true,
        }),
        Animated.timing(scaleValue, {
          toValue: 1,
          duration: 1000,
          easing: Easing.inOut(Easing.ease),
          useNativeDriver: true,
        }),
      ]),
      {
        iterations: -1, // -1 means infinite loop
      }
    );
  };

  createLoopAnimation().start();
};

// Interpolation values for smooth transitions
const slideTranslateX = slideValue.interpolate({
  inputRange: [0, 1],
  outputRange: [0, 200], // Move 200 pixels to the right
});

```

```

const rotateInterpolate = rotateValue.interpolate({
  inputRange: [0, 1],
  outputRange: ['0deg', '360deg'], // Full rotation
});

const colorInterpolate = colorValue.interpolate({
  inputRange: [0, 1],
  outputRange: ['#3498db', '#e74c3c'], // Blue to red
});

const scaleInterpolate = scaleValue.interpolate({
  inputRange: [1, 1.5],
  outputRange: [1, 1.5],
  extrapolate: 'clamp', // Prevent values outside range
});

return (
  <View style={styles.container}>
    <Text style={styles.title}>React Native Animations</Text>

    {/* Animation Control Buttons */}
    <View style={styles.buttonContainer}>
      <TouchableOpacity style={styles.button} onPress={fadeInOut}>
        <Text style={styles.buttonText}>Fade In/Out</Text>
      </TouchableOpacity>

      <TouchableOpacity style={styles.button} onPress={slideAnimation}>
        <Text style={styles.buttonText}>Slide</Text>
      </TouchableOpacity>

      <TouchableOpacity style={styles.button} onPress={scaleAnimation}>
        <Text style={styles.buttonText}>Scale Spring</Text>
      </TouchableOpacity>

      <TouchableOpacity style={styles.button} onPress={rotateAnimation}>
        <Text style={styles.buttonText}>Rotate</Text>
      </TouchableOpacity>

      <TouchableOpacity style={styles.button} onPress={colorAnimation}>
        <Text style={styles.buttonText}>Color Change</Text>
      </TouchableOpacity>

      <TouchableOpacity
        style={[styles.button, styles.complexButton]}
        onPress={complexAnimation}
        disabled={isAnimating}
      >
        <Text style={styles.buttonText}>

```

```

        {isAnimating ? 'Animating...' : 'Complex Animation'}
      </Text>
    </TouchableOpacity>

    <TouchableOpacity style={styles.button} onPress={startLoopAnimation}>
      <Text style={styles.buttonText}>Start Loop</Text>
    </TouchableOpacity>
  </View>

  {/ * Animation Playground */}
  <View style={styles.playground}>
    {/ * Fade Animation Box */}
    <Animated.View
      style={[
        styles.animatedBox,
        styles.fadeBox,
        {
          opacity: fadeValue,
        },
      ]}
    >
      <Text style={styles.boxText}>Fade</Text>
    </Animated.View>

    {/ * Slide Animation Box */}
    <Animated.View
      style={[
        styles.animatedBox,
        styles.slideBox,
        {
          transform: [{ translateX: slideTranslateX }],
        },
      ]}
    >
      <Text style={styles.boxText}>Slide</Text>
    </Animated.View>

    {/ * Scale Animation Box */}
    <Animated.View
      style={[
        styles.animatedBox,
        styles.scaleBox,
        {
          transform: [{ scale: scaleValue }],
        },
      ]}
    >
      <Text style={styles.boxText}>Scale</Text>
    </Animated.View>

```

```
    { /* Rotate Animation Box */ }
    <Animated.View
      style={
        [
          styles.animatedBox,
          styles.rotateBox,
          {
            transform: [{ rotate: rotateInterpolate }],
          },
        ]
      }
    >
      <Text style={styles.boxText}>Rotate</Text>
    </Animated.View>

    { /* Color Animation Box */ }
    <Animated.View
      style={
        [
          styles.animatedBox,
          {
            backgroundColor: colorInterpolate,
          },
        ]
      }
    >
      <Text style={styles.boxText}>Color</Text>
    </Animated.View>

    { /* Draggable Box with Pan Gesture */ }
    <PanGestureHandler
      onGestureEvent={onPanGestureEvent}
      onHandlerStateChange={onPanHandlerStateChange}
    >
      <Animated.View
        style={
          [
            styles.animatedBox,
            styles.draggableBox,
            {
              transform: [
                { translateX: panValue.x },
                { translateY: panValue.y },
              ],
            },
          ]
        }
      >
        <Text style={styles.boxText}>Drag Me</Text>
      </Animated.View>
    </PanGestureHandler>
  </View>

  { /* Animation Status */ }
```

```

    <View style={styles.statusContainer}>
      <Text style={styles.statusText}>
        Animation Status: {isAnimating ? 'Running' : 'Idle'}
      </Text>
    </View>
  </View>
);
}

```

```

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    backgroundColor: '#f8f9fa',
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    textAlign: 'center',
    marginBottom: 20,
    color: '#2c3e50',
  },
  buttonContainer: {
    flexDirection: 'row',
    flexWrap: 'wrap',
    justifyContent: 'space-between',
    marginBottom: 30,
  },
  button: {
    backgroundColor: '#3498db',
    paddingHorizontal: 12,
    paddingVertical: 8,
    borderRadius: 6,
    marginBottom: 8,
    minWidth: '48%',
    alignItems: 'center',
  },
  complexButton: {
    backgroundColor: '#e74c3c',
    minWidth: '100%',
  },
  buttonText: {
    color: 'white',
    fontSize: 12,
    fontWeight: '600',
  },
  playground: {
    flex: 1,
    justifyContent: 'space-around',
  },

```

```
    alignItems: 'center',
    paddingVertical: 20,
  },
  animatedBox: {
    width: 80,
    height: 80,
    borderRadius: 8,
    justifyContent: 'center',
    alignItems: 'center',
    marginVertical: 10,
  },
  fadeBox: {
    backgroundColor: '#9b59b6',
  },
  slideBox: {
    backgroundColor: '#2ecc71',
  },
  scaleBox: {
    backgroundColor: '#f39c12',
  },
  rotateBox: {
    backgroundColor: '#1abc9c',
  },
  draggableBox: {
    backgroundColor: '#34495e',
  },
  boxText: {
    color: 'white',
    fontSize: 12,
    fontWeight: 'bold',
    textAlign: 'center',
  },
  statusContainer: {
    alignItems: 'center',
    paddingVertical: 10,
  },
  statusText: {
    fontSize: 14,
    color: '#7f8c8d',
  },
});
```

## Chapter 11: Performance Optimization

## Understanding React Native Performance

Performance optimization is critical for creating smooth, responsive mobile applications. React Native apps can suffer from various performance bottlenecks, but understanding these issues and their solutions is key to building production-ready apps.

### Common Performance Issues:

- **JavaScript Thread Blocking:** Heavy computations blocking UI updates
- **Memory Leaks:** Unreleased resources causing crashes
- **Unnecessary Re-renders:** Components updating when they shouldn't
- **Large Bundle Sizes:** Slow app startup and increased storage usage
- **Image Loading:** Unoptimized images consuming excessive memory

### Performance Monitoring Tools:

- **React DevTools:** Component render analysis
- **Flipper:** Comprehensive debugging and profiling
- **Metro Bundle Analyzer:** Bundle size analysis
- **Native Profilers:** Xcode Instruments and Android Profiler

```
import React, { useState, useEffect, useMemo, useCallback, memo } from 'react';
import {
  View,
  Text,
  FlatList,
  Image,
  TouchableOpacity,
  ActivityIndicator,
  Alert,
  StyleSheet,
} from 'react-native';

/**
 * Optimized List Item Component
 * Using React.memo to prevent unnecessary re-renders
 * This component only re-renders when its props actually change
 */
const OptimizedListItem = memo(({ item, onPress, isSelected }) => {
  console.log(`Rendering item: ${item.id}`); // Debug log to track renders

  return (
    <TouchableOpacity
      style={[
        styles.listItem,
        isSelected && styles.selectedItem
      ]}
      onPress={() => onPress(item.id)}
    >
```

 Copy

```

    activeOpacity={0.7}
  >
  {/** Optimized Image Loading */}
  <Image
    source={{ uri: item.imageUrl }}
    style={styles.itemImage}
    resizeMode="cover"
    // Performance optimizations for images
    loadingIndicatorSource={require('./assets/loading-placeholder.png')}
    defaultSource={require('./assets/default-image.png')}
    // Fade in animation when image loads
    fadeDuration={300}
  />

  <View style={styles.itemContent}>
    <Text style={styles.itemTitle} numberOfLines={2}>
      {item.title}
    </Text>
    <Text style={styles.itemDescription} numberOfLines={3}>
      {item.description}
    </Text>
    <View style={styles.itemFooter}>
      <Text style={styles.itemPrice}>${item.price}</Text>
      <Text style={styles.itemCategory}>{item.category}</Text>
    </View>
  </View>
</TouchableOpacity>
);
});

/**
 * Main Performance Demo Component
 * Demonstrates various optimization techniques
 */
export default function PerformanceExample() {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(false);
  const [selectedItems, setSelectedItems] = useState(new Set());
  const [filter, setFilter] = useState('all');
  const [searchQuery, setSearchQuery] = useState('');

  /**
   * Generate large dataset for performance testing
   * This simulates loading data from an API
   */
  const generateLargeDataset = useCallback(() => {
    const categories = ['Electronics', 'Clothing', 'Books', 'Home', 'Sports'];
    const dataset = [];

```

```

    for (let i = 0; i < 1000; i++) {
      dataset.push({
        id: i.toString(),
        title: `Product ${i + 1}`,
        description: `This is a detailed description for product ${i + 1}. It
contains multiple lines of text to test rendering performance.`,
        price: (Math.random() * 500 + 10).toFixed(2),
        category: categories[i % categories.length],
        imageUrl: `https://picsum.photos/80/80?random=${i}`,
        rating: (Math.random() * 5).toFixed(1),
        inStock: Math.random() > 0.2, // 80% in stock
      });
    }

    return dataset;
  }, []);

/**
 * Load data with loading state management
 * Simulates API call with proper loading states
 */
const loadData = useCallback(async () => {
  setLoading(true);

  try {
    // Simulate network delay
    await new Promise(resolve => setTimeout(resolve, 1000));

    const newData = generateLargeDataset();
    setData(newData);
  } catch (error) {
    Alert.alert('Error', 'Failed to load data');
  } finally {
    setLoading(false);
  }
}, [generateLargeDataset]);

/**
 * Optimized filtered data using useMemo
 * This prevents recalculating filtered data on every render
 * Only recalculates when data, filter, or searchQuery changes
 */
const filteredData = useMemo(() => {
  console.log('Filtering data...'); // Debug log

  let filtered = data;

  // Apply category filter
  if (filter !== 'all') {

```

```

        filtered = filtered.filter(item =>
            item.category.toLowerCase() === filter.toLowerCase()
        );
    }

    // Apply search filter
    if (searchQuery.trim()) {
        const query = searchQuery.toLowerCase();
        filtered = filtered.filter(item =>
            item.title.toLowerCase().includes(query) ||
            item.description.toLowerCase().includes(query)
        );
    }

    return filtered;
}, [data, filter, searchQuery]);

/**
 * Optimized selection handler using useCallback
 * Prevents creating new function on every render
 */
const handleItemPress = useCallback((itemId) => {
    setSelectedItems(prevSelected => {
        const newSelected = new Set(prevSelected);
        if (newSelected.has(itemId)) {
            newSelected.delete(itemId);
        } else {
            newSelected.add(itemId);
        }
        return newSelected;
    });
}, []);

/**
 * Batch operations for better performance
 * Group multiple state updates together
 */
const clearSelections = useCallback(() => {
    // Using functional update to avoid dependency on selectedItem
    setSelectedItems(new Set());
}, []);

/**
 * Optimized render function for FlatList
 * Using useCallback to prevent recreation on every render
 */
const renderItem = useCallback(({ item }) => (
    <OptimizedListItem
        item={item}

```

```

        onPress={handleItemPress}
        isSelected={selectedItems.has(item.id)}
      />
    ), [handleItemPress, selectedItems]);

  /**
   * Key extractor function (stable reference)
   */
  const keyExtractor = useCallback((item) => item.id, []);

  /**
   * Item separator component (memoized)
   */
  const ItemSeparator = memo(() => <View style={styles.separator} />);

  /**
   * Footer component for loading indicator
   */
  const ListFooter = memo(() => {
    if (!loading) return null;

    return (
      <View style={styles.footerLoader}>
        <ActivityIndicator size="large" color="#3498db" />
        <Text style={styles.loadingText}>Loading more items...</Text>
      </View>
    );
  });

  /**
   * Header component with statistics
   */
  const ListHeader = memo(() => (
    <View style={styles.listHeader}>
      <Text style={styles.statsText}>
        Showing {filteredData.length} of {data.length} items
      </Text>
      <Text style={styles.statsText}>
        Selected: {selectedItems.size}
      </Text>
    </View>
  ));

  /**
   * Load data when component mounts
   */
  useEffect(() => {
    loadData();
  }, [loadData]);

```

```

/**
 * Performance monitoring hook
 * Track render times and memory usage
 */
useEffect(() => {
  const startTime = performance.now();

  return () => {
    const endTime = performance.now();
    console.log(`Component render time: ${endTime - startTime}ms`);
  };
});

/**
 * Memory cleanup
 * Clear large objects when component unmounts
 */
useEffect(() => {
  return () => {
    setData([]);
    setSelectedItems(new Set());
  };
}, []);

if (loading && data.length === 0) {
  return (
    <View style={styles.centerContainer}>
      <ActivityIndicator size="large" color="#3498db" />
      <Text style={styles.loadingText}>Loading data...</Text>
    </View>
  );
}

return (
  <View style={styles.container}>
    <Text style={styles.title}>Performance Optimization Demo</Text>

    {/* Performance Stats */}
    <View style={styles.statsContainer}>
      <Text style={styles.statsTitle}>Performance Stats:</Text>
      <Text style={styles.statsText}>• Using React.memo for list items</Text>
      <Text style={styles.statsText}>• useMemo for filtered data</Text>
      <Text style={styles.statsText}>• useCallback for event handlers</Text>
      <Text style={styles.statsText}>• FlatList with optimizations</Text>
    </View>

    {/* Filter Controls */}
    <View style={styles.filterContainer}>

```

```

<Text style={styles.filterTitle}>Filter:</Text>
{['all', 'Electronics', 'Clothing', 'Books'].map((filterOption) => (
  <TouchableOpacity
    key={filterOption}
    style={[
      styles.filterButton,
      filter === filterOption && styles.activeFilter,
    ]}
    onPress={() => setFilter(filterOption)}
  >
    <Text style={[
      styles.filterButtonText,
      filter === filterOption && styles.activeFilterText,
    ]}>
      {filterOption}
    </Text>
  </TouchableOpacity>
)
)}}
</View>

{/* Optimized FlatList */}
<FlatList
  data={filteredData}
  renderItem={renderItem}
  keyExtractor={keyExtractor}
  ListHeaderComponent={ListHeader}
  ListFooterComponent={ListFooter}
  ItemSeparatorComponent={ItemSeparator}

  // Performance optimizations
  removeClippedSubviews={true} // Remove off-screen items from memory
  maxToRenderPerBatch={10} // Render 10 items per batch
  windowSize={10} // Keep 10 screens worth of items in memory
  initialNumToRender={15} // Render 15 items initially
  updateCellsBatchingPeriod={50} // Batch updates every 50ms

  // Optimize scrolling performance
  scrollEventThrottle={16} // Throttle scroll events to 60fps
  showsVerticalScrollIndicator={false}

  // GetItemLayout for known item heights (major performance boost)
  getItemLayout={(data, index) => ({
    length: 120, // Fixed item height
    offset: 120 * index,
    index,
  })}

  // Refresh control
  onRefresh={loadData}

```

```

        refreshing={loading}
      />

      {//* Action Buttons */}
      {selectedItems.size > 0 && (
        <View style={styles.actionContainer}>
          <TouchableOpacity
            style={styles.clearButton}
            onPress={clearSelections}
          >
            <Text style={styles.buttonText}>
              Clear Selection ({selectedItems.size})
            </Text>
          </TouchableOpacity>
        </View>
      )}
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f8f9fa',
  },
  centerContainer: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    textAlign: 'center',
    padding: 15,
    color: '#2c3e50',
  },
  statsContainer: {
    backgroundColor: '#ecf0f1',
    padding: 15,
    marginHorizontal: 10,
    borderRadius: 8,
    marginBottom: 10,
  },
  statsTitle: {
    fontSize: 16,
    fontWeight: 'bold',
    marginBottom: 8,
    color: '#2c3e50',
  },

```

```
    },
    statsText: {
      fontSize: 12,
      color: '#34495e',
      marginBottom: 2,
    },
    filterContainer: {
      flexDirection: 'row',
      alignItems: 'center',
      paddingHorizontal: 15,
      paddingVertical: 10,
      backgroundColor: 'white',
      marginBottom: 5,
    },
    filterTitle: {
      fontSize: 14,
      fontWeight: '600',
      marginRight: 10,
      color: '#2c3e50',
    },
    filterButton: {
      paddingHorizontal: 12,
      paddingVertical: 6,
      borderRadius: 15,
      backgroundColor: '#ecf0f1',
      marginRight: 8,
    },
    activeFilter: {
      backgroundColor: '#3498db',
    },
    filterButtonText: {
      fontSize: 12,
      color: '#7f8c8d',
    },
    activeFilterText: {
      color: 'white',
    },
    listHeader: {
      flexDirection: 'row',
      justifyContent: 'space-between',
      padding: 15,
      backgroundColor: 'white',
    },
    listItem: {
      flexDirection: 'row',
      padding: 15,
      backgroundColor: 'white',
      alignItems: 'center',
    },
  },
```

```
selectedItem: {
  backgroundColor: '#ebf3fd',
},
itemImage: {
  width: 80,
  height: 80,
  borderRadius: 8,
  marginRight: 15,
},
itemContent: {
  flex: 1,
},
itemTitle: {
  fontSize: 16,
  fontWeight: 'bold',
  color: '#2c3e50',
  marginBottom: 4,
},
itemDescription: {
  fontSize: 14,
  color: '#7f8c8d',
  lineHeight: 1.8,
  marginBottom: 8,
},
itemFooter: {
  flexDirection: 'row',
  justifyContent: 'space-between',
  alignItems: 'center',
},
itemPrice: {
  fontSize: 16,
  fontWeight: 'bold',
  color: '#e74c3c',
},
itemCategory: {
  fontSize: 12,
  color: '#3498db',
  backgroundColor: '#ecf0f1',
  paddingHorizontal: 8,
  paddingVertical: 2,
  borderRadius: 10,
},
separator: {
  height: 1,
  backgroundColor: '#ecf0f1',
  marginHorizontal: 15,
},
footerLoader: {
  padding: 20,
```

```
      alignItems: 'center',
    },
    loadingText: {
      marginTop: 10,
      color: '#7f8c8d',
    },
    actionContainer: {
      padding: 15,
      backgroundColor: 'white',
      borderTopWidth: 1,
      borderTopColor: '#ecf0f1',
    },
    clearButton: {
      backgroundColor: '#e74c3c',
      padding: 12,
      borderRadius: 8,
      alignItems: 'center',
    },
    buttonText: {
      color: 'white',
      fontSize: 16,
      fontWeight: '600',
    },
  },
});
```

## Chapter 12: Testing and Debugging

### Testing Strategy for React Native

Testing is essential for maintaining code quality and preventing regressions. React Native testing involves multiple layers: unit tests, integration tests, and end-to-end tests. Understanding different testing approaches and tools is crucial for building reliable applications.

#### Testing Pyramid:

- **Unit Tests:** Test individual functions and components
- **Integration Tests:** Test component interactions
- **End-to-End Tests:** Test complete user workflows
- **Visual Regression Tests:** Ensure UI consistency

#### Testing Tools:

- **Jest:** JavaScript testing framework (built-in with React Native)
- **React Native Testing Library:** Component testing utilities
- **Detox:** End-to-end testing framework

- **Flipper:** Debugging and inspection tool

Copy

```
// __tests__/UserProfile.test.js
import React from 'react';
import { render, fireEvent, waitFor } from '@testing-library/react-native';
import AsyncStorage from '@react-native-async-storage/async-storage';
import UserProfile from '../components/UserProfile';

// Mock AsyncStorage
jest.mock('@react-native-async-storage/async-storage', () => ({
  getItem: jest.fn(),
  setItem: jest.fn(),
  removeItem: jest.fn(),
}));

// Mock navigation
const mockNavigation = {
  navigate: jest.fn(),
  goBack: jest.fn(),
  replace: jest.fn(),
};

/**
 * Test Suite for UserProfile Component
 * Demonstrates comprehensive component testing
 */
describe('UserProfile Component', () => {
  // Clear all mocks before each test
  beforeEach(() => {
    jest.clearAllMocks();
  });

  /**
   * Test initial render and loading state
   */
  it('should render loading state initially', () => {
    const { getByTestId, getByText } = render(
      <UserProfile navigation={mockNavigation} />
    );

    // Check if loading indicator is visible
    expect(getByTestId('loading-indicator')).toBeTruthy();
    expect(getByText('Loading user profile...')).toBeTruthy();
  });

  /**
   * Test successful user data loading
   */
}
```

```
it('should display user data after loading', async () => {
  // Mock AsyncStorage response
  const mockUserData = {
    id: '1',
    name: 'John Doe',
    email: 'john@example.com',
    profilePicture: 'https://example.com/avatar.jpg',
  };

  AsyncStorage.getItem.mockResolvedValueOnce(JSON.stringify(mockUserData));

  const { getByText, getByTestId, queryByTestId } = render(
    <UserProfile navigation={mockNavigation} />
  );

  // Wait for loading to complete
  await waitFor(() => {
    expect(queryByTestId('loading-indicator')).toBeNull();
  });

  // Check if user data is displayed
  expect(getByText('John Doe')).toBeTruthy();
  expect(getByText('john@example.com')).toBeTruthy();
  expect(getByTestId('profile-picture')).toBeTruthy();
});

/**
 * Test edit functionality
 */
it('should allow editing user profile', async () => {
  const mockUserData = {
    id: '1',
    name: 'John Doe',
    email: 'john@example.com',
  };

  AsyncStorage.getItem.mockResolvedValueOnce(JSON.stringify(mockUserData));

  const { getByText, getByTestId, getByDisplayValue } = render(
    <UserProfile navigation={mockNavigation} />
  );

  // Wait for data to load
  await waitFor(() => {
    expect(getByText('John Doe')).toBeTruthy();
  });

  // Tap edit button
  fireEvent.press(getByTestId('edit-button'));
```

```

// Check if edit mode is activated
expect(getByTestId('name-input')).toBeTruthy();
expect(getByDisplayValue('John Doe')).toBeTruthy();

// Change name
fireEvent.changeText(getByTestId('name-input'), 'Jane Doe');

// Save changes
fireEvent.press(getByTestId('save-button'));

// Verify AsyncStorage.setItem was called
await waitFor(() => {
  expect(AsyncStorage.setItem).toHaveBeenCalledWith(
    '@user_profile',
    JSON.stringify({
      ...mockUserData,
      name: 'Jane Doe',
    })
  );
});

/**
 * Test error handling
 */
it('should handle loading errors gracefully', async () => {
  // Mock AsyncStorage to throw an error
  AsyncStorage.getItem.mockRejectedValueOnce(new Error('Storage error'));

  const { getByText, queryByTestId } = render(
    <UserProfile navigation={mockNavigation} />
  );

  // Wait for error to be handled
  await waitFor(() => {
    expect(queryByTestId('loading-indicator')).toBeNull();
  });

  // Check if error message is displayed
  expect(getByText('Failed to load user profile')).toBeTruthy();
  expect(getByText('Retry')).toBeTruthy();
});

/**
 * Test navigation functionality
 */
it('should navigate to settings when settings button is pressed', async () => {
  const mockUserData = {

```

```
    id: '1',
    name: 'John Doe',
    email: 'john@example.com',
  };

AsyncStorage.getItem.mockResolvedValueOnce(JSON.stringify(mockUserData));

const { getByTestId } = render(
  <UserProfile navigation={mockNavigation} />
);

// Wait for component to load
await waitFor(() => {
  expect(getByTestId('settings-button')).toBeTruthy();
});

// Press settings button
fireEvent.press(getByTestId('settings-button'));

// Verify navigation was called
expect(mockNavigation.navigate).toHaveBeenCalledWith('Settings');
});
});

/**
 * Test Suite for Utility Functions
 * Demonstrates unit testing of helper functions
 */
describe('Profile Utilities', () => {
  /**
   * Test email validation function
   */
  describe('validateEmail', () => {
    it('should return true for valid emails', () => {
      const validEmails = [
        'test@example.com',
        'user.name@domain.co.uk',
        'user+tag@example.org',
      ];

      validEmails.forEach(email => {
        expect(validateEmail(email)).toBe(true);
      });
    });

    it('should return false for invalid emails', () => {
      const invalidEmails = [
        'invalid-email',
        '@example.com',
      ];
    });
  });
});
```

```
        'user@',
        'user space@example.com',
        '',
    ];

    invalidEmails.forEach(email => {
        expect(validateEmail(email)).toBe(false);
    });
});

/**
 * Test profile data formatting
 */
describe('formatProfileData', () => {
    it('should format profile data correctly', () => {
        const rawData = {
            first_name: 'John',
            last_name: 'Doe',
            email_address: 'john@example.com',
            profile_image_url: 'https://example.com/image.jpg',
        };

        const expected = {
            name: 'John Doe',
            email: 'john@example.com',
            profilePicture: 'https://example.com/image.jpg',
        };

        expect(formatProfileData(rawData)).toEqual(expected);
    });

    it('should handle missing data gracefully', () => {
        const rawData = {
            first_name: 'John',
            // missing last_name and other fields
        };

        const result = formatProfileData(rawData);

        expect(result.name).toBe('John');
        expect(result.email).toBe('');
        expect(result.profilePicture).toBe('');
    });
});

/**
 * Integration Test Suite
```

```
* Tests component interactions and data flow
*/
describe('UserProfile Integration', () => {
  it('should update user data and reflect changes immediately', async () => {
    // Mock initial user data
    const initialUserData = {
      id: '1',
      name: 'John Doe',
      email: 'john@example.com',
    };

    AsyncStorage.getItem.mockResolvedValueOnce(JSON.stringify(initialUserData));

    const { getByText, getByTestId, getByDisplayValue, rerender } = render(
      <UserProfile navigation={mockNavigation} />
    );

    // Wait for initial load
    await waitFor(() => {
      expect(getByText('John Doe')).toBeTruthy();
    });

    // Enter edit mode
    fireEvent.press(getByTestId('edit-button'));

    // Update name
    fireEvent.changeText(getByTestId('name-input'), 'Jane Smith');
    fireEvent.changeText(getByTestId('email-input'), 'jane@example.com');

    // Save changes
    fireEvent.press(getByTestId('save-button'));

    // Verify the component shows updated data
    await waitFor(() => {
      expect(getByText('Jane Smith')).toBeTruthy();
      expect(getByText('jane@example.com')).toBeTruthy();
    });

    // Verify data was saved to AsyncStorage
    expect(AsyncStorage.setItem).toHaveBeenCalledWith(
      '@user_profile',
      JSON.stringify({
        id: '1',
        name: 'Jane Smith',
        email: 'jane@example.com',
      })
    );
  });
});
```

```
/**
 * Test network failure and retry mechanism
 */
it('should handle network failures and allow retry', async () => {
  // First call fails
  AsyncStorage.getItem.mockRejectedValueOnce(new Error('Network error'));

  const { getByText, getByTestId, rerender } = render(
    <UserProfile navigation={mockNavigation} />
  );

  // Wait for error state
  await waitFor(() => {
    expect(getByText('Failed to load user profile')).toBeTruthy();
  });

  // Clear the mock and set up successful response
  AsyncStorage.getItem.mockClear();
  AsyncStorage.getItem.mockResolvedValueOnce(JSON.stringify({
    id: '1',
    name: 'John Doe',
    email: 'john@example.com',
  }));

  // Press retry button
  fireEvent.press(getByTestId('retry-button'));

  // Wait for successful load
  await waitFor(() => {
    expect(getByText('John Doe')).toBeTruthy();
  });
});

/**
 * Performance Testing
 * Tests for component rendering performance
 */
describe('UserProfile Performance', () => {
  it('should not re-render unnecessarily', () => {
    const renderSpy = jest.fn();

    const TestComponent = () => {
      renderSpy();
      return <UserProfile navigation={mockNavigation} />;
    };

    const { rerender } = render(<TestComponent />);
```

```
// Initial render
expect(renderSpy).toHaveBeenCalledTimes(1);

// Re-render with same props
rerender(<TestComponent />);

// Should not trigger additional renders if props haven't changed
expect(renderSpy).toHaveBeenCalledTimes(1);
});
});

/**
 * Accessibility Testing
 * Ensures components are accessible
 */
describe('UserProfile Accessibility', () => {
  it('should have proper accessibility labels', async () => {
    const mockUserData = {
      id: '1',
      name: 'John Doe',
      email: 'john@example.com',
    };

    AsyncStorage.getItem.mockResolvedValueOnce(JSON.stringify(mockUserData));

    const { getByLabelText } = render(
      <UserProfile navigation={mockNavigation} />
    );

    await waitFor(() => {
      expect(getByLabelText('User name')).toBeTruthy();
      expect(getByLabelText('Email address')).toBeTruthy();
      expect(getByLabelText('Edit profile')).toBeTruthy();
    });
  });

  it('should support screen readers', async () => {
    const mockUserData = {
      id: '1',
      name: 'John Doe',
      email: 'john@example.com',
    };

    AsyncStorage.getItem.mockResolvedValueOnce(JSON.stringify(mockUserData));

    const { getByA11yRole } = render(
      <UserProfile navigation={mockNavigation} />
    );
  });
});
```

```

    await waitFor(() => {
      expect(getByA11yRole('button')).toBeTruthy(); // Edit button
      expect(getByA11yRole('text')).toBeTruthy(); // User info
    });
  });
});

```

## Debugging Techniques and Tools

 Copy

```

// utils/debugUtils.js
/**
 * Debugging Utilities for React Native
 * Collection of tools and techniques for effective debugging
 */

/**
 * Performance monitoring utility
 * Tracks component render times and identifies bottlenecks
 */
export const usePerformanceMonitor = (componentName) => {
  useEffect(() => {
    const startTime = performance.now();

    return () => {
      const endTime = performance.now();
      const renderTime = endTime - startTime;

      if (renderTime > 16) { // More than one frame (60fps)
        console.warn(`🐢 Slow render detected in ${componentName}:
${renderTime.toFixed(2)}ms`);
      }

      console.log(`🕒 ${componentName} render time: ${renderTime.toFixed(2)}ms`);
    };
  });
};

/**
 * Memory usage tracker
 * Monitors memory consumption and warns about potential leaks
 */
export const useMemoryMonitor = () => {
  useEffect(() => {
    const logMemoryUsage = () => {
      if (global.performance && global.performance.memory) {
        const memory = global.performance.memory;
        const used = Math.round(memory.usedJSHeapSize / 1048576 * 100) / 100;

```

```

const total = Math.round(memory.totalJSHeapSize / 1048576 * 100) / 100;

console.log(`📄 Memory usage: ${used}MB / ${total}MB`);

if (used / total > 0.9) {
  console.warn('⚠️ High memory usage detected!');
}
}
};

const interval = setInterval(logMemoryUsage, 10000); // Check every 10 seconds

return () => clearInterval(interval);
}, []);
};

/**
 * Network request debugger
 * Logs all network requests with timing information
 */
export const debugNetworkRequests = () => {
  const originalFetch = global.fetch;

  global.fetch = async (url, options = {}) => {
    const startTime = Date.now();

    console.log(`🌐 Starting request to: ${url}`);
    console.log('📄 Request options:', options);

    try {
      const response = await originalFetch(url, options);
      const endTime = Date.now();

      console.log(`✅ Request completed in ${endTime - startTime}ms`);
      console.log(`📊 Status: ${response.status} ${response.statusText}`);

      return response;
    } catch (error) {
      const endTime = Date.now();

      console.error(`❌ Request failed after ${endTime - startTime}ms`);
      console.error('💣 Error:', error);

      throw error;
    }
  };
};

/**

```

```

* Redux state debugger
* Logs state changes with before/after comparison
*/
export const createDebugMiddleware = () => (store) => (next) => (action) => {
  console.group(`🔍 Action: ${action.type}`);
  console.log('📄 Payload:', action.payload);

  const prevState = store.getState();
  console.log('📄 Previous state:', prevState);

  const result = next(action);

  const nextState = store.getState();
  console.log('📄 Next state:', nextState);

  // Find what changed
  const changes = findStateChanges(prevState, nextState);
  if (changes.length > 0) {
    console.log('🔍 Changes:', changes);
  }

  console.groupEnd();

  return result;
};

/**
 * Find differences between two state objects
 */
const findStateChanges = (prevState, nextState, path = '') => {
  const changes = [];

  for (const key in nextState) {
    const currentPath = path ? `${path}.${key}` : key;

    if (prevState[key] !== nextState[key]) {
      if (typeof nextState[key] === 'object' && nextState[key] !== null) {
        changes.push(...findStateChanges(prevState[key] || {}, nextState[key],
currentPath));
      } else {
        changes.push({
          path: currentPath,
          from: prevState[key],
          to: nextState[key],
        });
      }
    }
  }
}

```

```

    return changes;
  };

  /**
   * Component lifecycle debugger
   * Tracks component mount/unmount cycles
   */
  export const useLifecycleDebugger = (componentName) => {
    useEffect(() => {
      console.log(`🏠 ${componentName} mounted`);

      return () => {
        console.log(`🏠 ${componentName} unmounted`);
      };
    }, [componentName]);

    useEffect(() => {
      console.log(`🔄 ${componentName} updated`);
    });
  };

  /**
   * Async operation tracker
   * Monitors promises and async operations
   */
  export const trackAsyncOperation = async (name, operation) => {
    console.log(`⌚ Starting async operation: ${name}`);
    const startTime = Date.now();

    try {
      const result = await operation();
      const endTime = Date.now();

      console.log(`✅ Async operation "${name}" completed in ${endTime -
startTime}ms`);
      return result;
    } catch (error) {
      const endTime = Date.now();

      console.error(`❌ Async operation "${name}" failed after ${endTime -
startTime}ms`);
      console.error('💣 Error:', error);

      throw error;
    }
  };

  /**
   * Custom hook for debugging props changes

```

```

*/
export const useWhyDidYouUpdate = (name, props) => {
  // Get a mutable ref object where we can store props for comparison next time
  const previousProps = useRef();

  useEffect(() => {
    if (previousProps.current) {
      // Get all keys from previous and current props
      const allKeys = Object.keys({ ...previousProps.current, ...props });

      // Use this object to keep track of changed props
      const changedProps = {};

      // Iterate through keys
      allKeys.forEach((key) => {
        // If previous is different from current
        if (previousProps.current[key] !== props[key]) {
          // Add to changedProps
          changedProps[key] = {
            from: previousProps.current[key],
            to: props[key],
          };
        }
      });

      // If changedProps after iteration has keys then output them
      if (Object.keys(changedProps).length) {
        console.log(`🔍 [${name}] Props that changed:`, changedProps);
      }
    }

    // Finally update previousProps with current props for next hook call
    previousProps.current = props;
  });
};

```

This completes **Part 3: Advanced Features** of the React Native course, covering:

**Chapter 9: Native Device Features** - Camera integration, location services, and local storage with comprehensive examples and error handling.

**Chapter 10: Animations and Advanced UI** - Various animation techniques including timing, spring, gesture-based, and complex animations with performance considerations.

**Chapter 11: Performance Optimization** - Memory management, render optimization, FlatList optimizations, and performance monitoring techniques.

**Chapter 12: Testing and Debugging** - Comprehensive testing strategies, unit tests, integration tests, and debugging utilities for maintaining code quality.

Each chapter provides detailed explanations of concepts, real-world code examples with extensive comments, and best practices for professional React Native development. The code examples demonstrate production-ready patterns with proper error handling, performance optimization, and user experience considerations.

Ready for **Part 4: Professional Development (Expert Level)** when you want to continue with advanced topics like Redux, authentication, push notifications, and deployment strategies!